

TECHNICAL REFERENCE GUIDE

# Azure SQL Star Schema Bulk Load & Data Modelling

EmployeeDatabase • GenFlow\_Validation Project

70 Million Row Fact Table • PK/FK Verified

Azure SQL Database

Star Schema

T-SQL

Python + pyodbc

DBeaver

Prepared: June 2026 • GitHub Reference Document

## Table of Contents

---

1. Star Schema — Kya hota hai aur kyun?

2. Database Schema (Tables, PK, FK)

3. DBeaver mein Azure SQL connect karna

4. Tables banana — 01\_create\_tables.sql

5. Mock Dimension Data — Python (Faker)

6. CSV ko DBeaver se Import karna

7. 70M Fact Rows — T-SQL Server-Side Load

8. 70M Fact Rows — Python + pyodbc CSV Upload

9. Issues aur Solutions (Troubleshooting)

10. PK/FK aur Join Verification

11. Azure Tier Scale Up/Down

12. Quick Reference Card

# 1. Star Schema — Kya hota hai aur kyun?

Star schema ek data warehouse design pattern hai jisme ek central **Fact Table** hoti hai jo measurable events store karti hai, aur uske around **Dimension Tables** hoti hain jo descriptive context deti hain. Diagram mein ye star jaisa dikhta hai — isliye naam Star Schema.

## Fact Table vs Dimension Table

| Cheez                | Fact Table                              | Dimension Table                        |
|----------------------|-----------------------------------------|----------------------------------------|
| Kya store karta hai? | Measurable events (sales, transactions) | Descriptive context (kaun, kahan, kab) |
| Size                 | Bahut bada (laakhon/crore rows)         | Chhota (kuch sau/hazaar rows)          |
| Keys                 | Foreign Keys (dim ki taraf point)       | Primary Key (unique)                   |
| Example              | Fact_Sales (70M rows)                   | Dim_Store, Dim_Product, Dim_Date       |

## PK aur FK ka concept

**Primary Key (PK):** Ek table mein har row ki unique pehchaan. Duplicate ya NULL allowed nahi. Example: `store_id` in `Dim_Store` — har store ka apna unique id.

**Foreign Key (FK):** Ek table ka column jo doosri table ke PK ko point karta hai. Example: `Fact_Sales.store_id` → `Dim_Store.store_id`. Agar FK mein koi value hai jo dim mein nahi — DB reject kar deta hai.

Key Rule

Fact table ki FK value dim table ki PK mein zaroor honi chahiye. Isse "Referential Integrity" kehte hain. Agar ye toot jaaye = data mismatch = galat analytics.

## Hamara Star Schema



## 2. Database Schema

### Fact\_Sales

#### Fact\_Sales

sale\_id BIGINT IDENTITY(1,1) **PK**

date\_key INT NOT NULL **FK**

store\_id INT NOT NULL **FK**

product\_id INT NOT NULL **FK**

payment\_mode VARCHAR(20)

quantity INT

unit\_price DECIMAL(10,2)

discount DECIMAL(10,2)

total\_amount DECIMAL(12,2)

### Dimension Tables

#### Dim\_Date

date\_key **PK**

full\_date

year

quarter

month

month\_name

#### Dim\_Store

store\_id **PK**

store\_type

region

state

city

#### Dim\_Product

product\_id **PK**

category

brand

### FK Relationships (Modelling Logic)

| Fact Column (FK)      | Points To | Dim Column (PK)        | Cardinality |
|-----------------------|-----------|------------------------|-------------|
| Fact_Sales.date_key   | →         | Dim_Date.date_key      | Many → One  |
| Fact_Sales.store_id   | →         | Dim_Store.store_id     | Many → One  |
| Fact_Sales.product_id | →         | Dim_Product.product_id | Many → One  |

### 3. DBeaver mein Azure SQL Connect Karna

#### Connection Details

| Field                  | Value                                  |
|------------------------|----------------------------------------|
| Host                   | research-db-pulse.database.windows.net |
| Port                   | 1433                                   |
| Database               | EmployeeDatabase                       |
| Authentication         | SQL Server Authentication              |
| Username               | Sql_server (NOT Sql_server@server)     |
| Encrypt                | true                                   |
| TrustServerCertificate | false                                  |

#### Steps

1. DBeaver kholo → Database → New Database Connection → SQL Server chuno.
2. Driver download maange to accept karo.
3. Main tab mein: Host, Port, Database, Username, Password bharo.
4. Driver Properties tab → `encrypt=true` , `trustServerCertificate=false` .
5. Test Connection → green tick aaye → Finish.

#### Common Issues

**Login failed:** Password mein { } braces mat lagao. Username sirf `Sql_server` (bina @server).

**Timeout/Cannot connect:** Azure portal mein firewall mein apna IP add karo: SQL Server → Networking → Add client IP → Save.

#### DBeaver mein Script Run Karna

| Script Type                 | Shortcut                                         | Kyun?                            |
|-----------------------------|--------------------------------------------------|----------------------------------|
| Simple SQL (CREATE, INSERT) | <code>Alt + X</code>                             | Statements alag-alag chalata hai |
| T-SQL with variables/loop   | <code>Ctrl+A</code> then <code>Ctrl+Enter</code> | Pura ek batch mein chalata hai   |

#### Important

Variables wale script (DECLARE, WHILE loop) ke liye hamesha **Ctrl+A → Ctrl+Enter**. Alt+X use karo to "Must declare scalar variable" error aata hai.

## 4. Tables Banana — 01\_create\_tables.sql

Ye script **DBeaver ke SQL editor** mein chalao ( **Alt+X** ).

```
-- Re-run safe: pehle Fact (FK wala), phir Dims drop
IF OBJECT_ID('dbo.Fact_Sales') IS NOT NULL DROP TABLE dbo.Fact_Sales;
IF OBJECT_ID('dbo.Dim_Date') IS NOT NULL DROP TABLE dbo.Dim_Date;
IF OBJECT_ID('dbo.Dim_Store') IS NOT NULL DROP TABLE dbo.Dim_Store;
IF OBJECT_ID('dbo.Dim_Product') IS NOT NULL DROP TABLE dbo.Dim_Product;

CREATE TABLE dbo.Dim_Date (
    date_key INT NOT NULL, full_date DATE, year SMALLINT,
    quarter VARCHAR(2), month TINYINT, month_name VARCHAR(15),
    CONSTRAINT PK_Dim_Date PRIMARY KEY (date_key)
);
CREATE TABLE dbo.Dim_Store (
    store_id INT NOT NULL, store_type VARCHAR(30), region VARCHAR(20),
    state VARCHAR(20), city VARCHAR(50),
    CONSTRAINT PK_Dim_Store PRIMARY KEY (store_id)
);
CREATE TABLE dbo.Dim_Product (
    product_id INT NOT NULL, category VARCHAR(50), brand VARCHAR(50),
    CONSTRAINT PK_Dim_Product PRIMARY KEY (product_id)
);
CREATE TABLE dbo.Fact_Sales (
    sale_id BIGINT IDENTITY(1,1) NOT NULL,
    date_key INT NOT NULL, store_id INT NOT NULL, product_id INT NOT NULL,
    payment_mode VARCHAR(20), quantity INT,
    unit_price DECIMAL(10,2), discount DECIMAL(10,2), total_amount DECIMAL(12,2),
    CONSTRAINT PK_Fact_Sales PRIMARY KEY (sale_id),
    CONSTRAINT FK_Fact_Date FOREIGN KEY (date_key) REFERENCES dbo.Dim_Date(date_key),
    CONSTRAINT FK_Fact_Store FOREIGN KEY (store_id) REFERENCES dbo.Dim_Store(store_id),
    CONSTRAINT FK_Fact_Product FOREIGN KEY (product_id) REFERENCES dbo.Dim_Product(product_id)
);
```

### Drop Order Rule

Hamesha pehle Fact\_Sales drop karo (FK wala table), phir Dimensions. Ulta karne se "FK constraint violation" error aata hai.

## 5. Mock Dimension Data — Python (Faker)

Ye script **VS Code / Terminal** mein chhalao (DBeaver mein nahi). Pehle: `pip install faker`

```
import csv, random
from datetime import date, timedelta
from faker import Faker
fake = Faker("en_IN"); random.seed(42); Faker.seed(42)

# Dim_Store: 200 unique stores
store_ids = random.sample(range(1000, 10000), 200)
REGIONS = {
    "North": [("DL", "Delhi"), ("RJ", "Jaipur"), ("UP", "Lucknow")],
    "West":  [("MH", "Mumbai"), ("GJ", "Ahmedabad"), ("MH", "Pune")],
    "South": [("KA", "Bengaluru"), ("TN", "Chennai"), ("TS", "Hyderabad")],
    "East":  [("WB", "Kolkata"), ("OD", "Bhubaneswar")],
}

with open("dim_store.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["store_id", "store_type", "region", "state", "city"])
    for sid in store_ids:
        region = random.choice(list(REGIONS))
        state, city = random.choice(REGIONS[region])
        w.writerow([sid, random.choice(["Kirana", "Supermarket", "Wholesale"]), region, state, city])

# Dim_Product: 500 unique products
product_ids = random.sample(range(100000, 1000000), 500)
CATS = {"Snacks": ["PepsiCo", "Haldiram", "ITC"],
        "Beverages": ["Coca-Cola", "Dabur"], "Dairy": ["Amul", "Nestle"]}

with open("dim_product.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["product_id", "category", "brand"])
    for pid in product_ids:
        cat = random.choice(list(CATS)); brand = random.choice(CATS[cat])
        w.writerow([pid, cat, brand])

# Dim_Date: full daily calendar 2022-2025
MONTHS=["January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"]

with open("dim_date.csv", "w", newline="") as f:
    w = csv.writer(f); w.writerow(["date_key", "full_date", "year", "quarter", "month", "month_name"])
    d = date(2022, 1, 1)
    while d <= date(2025, 12, 31):
        w.writerow([d.year*10000+d.month*100+d.day, d.isoformat(), d.year,
                    f"Q{(d.month-1)//3+1}", d.month, MONTHS[d.month-1]])
        d += timedelta(days=1)

print("3 CSVs ready: dim_store.csv, dim_product.csv, dim_date.csv")
```

## 6. CSV ko DBeaver se Import Karna

### Steps (har dimension table ke liye)

1. Left navigator: `EmployeeDatabase > Schemas > dbo > Tables`
2. `Dim_Store` pe right-click → **Import Data**
3. Format: **CSV** → Next
4. Browse karke `dim_store.csv` select karo → Next
5. Column mapping check karo — headers auto-match hote hain → Next
6. **Proceed** dabao
7. Yahi `Dim_Product` aur `Dim_Date` ke liye repeat karo

#### Important

**Fact\_Sales ke liye CSV import mat karo** — 3.3 GB CSV Azure pe upload karna slow aur unreliable hai. T-SQL ya pyodbc use karo (Sections 7/8).



## 7. 70M Fact Rows — T-SQL Server-Side Load

Sabse fast method — data Azure ke **andar hi** banta hai, 3.3 GB network pe transfer nahi hota. DBeaver mein **Ctrl+A** → **Ctrl+Enter** se chalo.

### Key Features

- **Resumable:** `@done = COUNT(*) FROM Fact_Sales` — jahan tak hua wahi se continue
- **FK off during load:** `NOCHECK CONSTRAINT ALL` → tez; end mein `WITH CHECK` validate
- **TABLOCK:** minimal logging → faster insert
- **Valid keys only:** dim tables ke keys temp tables mein load, sirf wahi use

```
SET NOCOUNT ON;
DECLARE @target bigint = 70000000; -- 7 crore
DECLARE @batch int = 2000000; -- 2M per loop
DECLARE @done bigint, @rows int, @nd int, @ns int, @np int;

ALTER TABLE dbo.Fact_Sales NOCHECK CONSTRAINT ALL; -- FK off

-- Dim keys ko numbered temp tables mein load karo
SELECT date_key, rn=ROW_NUMBER() OVER(ORDER BY(SELECT NULL)) INTO #dt FROM dbo.Dim_Date;
SELECT store_id, rn=ROW_NUMBER() OVER(ORDER BY(SELECT NULL)) INTO #st FROM dbo.Dim_Store;
SELECT product_id, rn=ROW_NUMBER() OVER(ORDER BY(SELECT NULL)) INTO #pr FROM dbo.Dim_Product;
SELECT @nd=COUNT(*) FROM #dt; SELECT @ns=COUNT(*) FROM #st; SELECT @np=COUNT(*) FROM #pr;
SELECT @done=COUNT_BIG(*) FROM dbo.Fact_Sales; -- resume point

WHILE @done < @target
BEGIN
    SET @rows=CASE WHEN @target-@done>@batch THEN @batch ELSE @target-@done END;
    ;WITH L0 AS(SELECT n=1 UNION ALL SELECT 1), L1 AS(SELECT n=1 FROM L0 a CROSS JOIN L0 b),
        L2 AS(SELECT n=1 FROM L1 a CROSS JOIN L1 b), L3 AS(SELECT n=1 FROM L2 a CROSS JOIN L2 b),
        L4 AS(SELECT n=1 FROM L3 a CROSS JOIN L3 b), L5 AS(SELECT n=1 FROM L4 a CROSS JOIN L4 b),
        Nums AS(SELECT TOP(@rows) g=ROW_NUMBER() OVER(ORDER BY(SELECT NULL)) FROM L5)
    INSERT INTO dbo.Fact_Sales WITH(TABLOCK)
        (date_key,store_id,product_id,payment_mode,quantity,unit_price,discount,total_amount)
    SELECT dt.date_key,st.store_id,pr.product_id,
        CHOOSE((CHECKSUM(NEWID())&0x7fffffff)%4+1,'Cash','Card','UPI','Wallet'),
        v.qty, v.price, v.disc, CAST(v.qty*v.price-v.disc AS decimal(12,2))
    FROM Nums
    CROSS APPLY(SELECT
        dr =(CHECKSUM(NEWID())&0x7fffffff)%@nd+1, sr=(CHECKSUM(NEWID())&0x7fffffff)%@ns+1,
        prn=(CHECKSUM(NEWID())&0x7fffffff)%@np+1,
        qty =(CHECKSUM(NEWID())&0x7fffffff)%50+1,
        price=CAST((CHECKSUM(NEWID())&0x7fffffff)%49000/100.0+10 AS decimal(10,2)),
        disc =CAST((CHECKSUM(NEWID())&0x7fffffff)%9000/100.0 AS decimal(10,2))) v
    JOIN #dt dt ON dt.rn=v.dr JOIN #st st ON st.rn=v.sr JOIN #pr pr ON pr.rn=v.prn;
    SET @done+=@rows;
    RAISERROR('Rows: %I64d',0,1,@done) WITH NOWAIT;
END

ALTER TABLE dbo.Fact_Sales WITH CHECK CHECK CONSTRAINT ALL; -- FK wapas on + validate
SELECT COUNT_BIG(*) AS fact_rows FROM dbo.Fact_Sales; -- 70000000
```

### Speed Tip

Azure tier scale up karne se speed kai guna badh jati hai. Load ke time bade tier pe jao, baad mein wapas chhota karo (Section 11).

## 8. 70M Fact Rows — Python + pyodbc CSV Upload

### Setup

```
# ODBC Driver install karo: https://aka.ms/odbc18
pip install pyodbc
```

### Script (upload\_fact\_sales.py)

```
import pyodbc, csv, time

SERVER="research-db-pulsee.database.windows.net"; DATABASE="EmployeeDatabase"
USERNAME="Sql_server"; PASSWORD="YOUR_PASSWORD"
CSV_FILE="C:/Users/Ayush/Downloads/mockdata/fact_sales.csv"
CHUNK_SIZE=10_000; TARGET=70_000_000; LOG_EVERY=100_000

CONN_STR=(f"DRIVER={{ODBC Driver 18 for SQL Server}};SERVER={SERVER},1433;"
          f"DATABASE={DATABASE};UID={USERNAME};PWD={PASSWORD};"
          f"Encrypt=yes;TrustServerCertificate=no;Connection Timeout=60;")
INSERT_SQL="""INSERT INTO dbo.Fact_Sales
(date_key,store_id,product_id,payment_mode,quantity,unit_price,discount,total_amount)
VALUES (?,?,?,?,?,?,?,?)"""

conn=pyodbc.connect(CONN_STR,autocommit=False); cur=conn.cursor()
cur.fast_executemany=True
cur.execute("ALTER TABLE dbo.Fact_Sales NOCHECK CONSTRAINT ALL"); conn.commit()
already=conn.cursor().execute("SELECT COUNT_BIG(*) FROM dbo.Fact_Sales").fetchone()[0]
print(f"Already: {already:}, Remaining: {TARGET-already:}")

with open(CSV_FILE,newline="",encoding="utf-8") as f:
    reader=csv.reader(f); next(reader) # header skip
    for _ in range(already): next(reader) # RESUME: already rows skip
    batch=[]; total=already; start=time.time()
    for row in reader:
        if total>=TARGET: break
        batch.append((int(row[0]),int(row[1]),int(row[2]),row[3],
                     int(row[4]),float(row[5]),float(row[6]),float(row[7])))
        if len(batch)>=CHUNK_SIZE:
            cur.executemany(INSERT_SQL,batch); conn.commit()
            total+=len(batch); batch=[]
        if total%LOG_EVERY==0:
            rate=(total-already)/(time.time()-start)
            print(f"[{total/TARGET*100:.1f}%] {total:}, | {rate:,.0f} r/s | ETA {(TARGET-total)/rate/60:.1f} min")

cur.execute("ALTER TABLE dbo.Fact_Sales WITH CHECK CHECK CONSTRAINT ALL"); conn.commit()
print(f"DONE: {conn.cursor().execute('SELECT COUNT_BIG(*) FROM dbo.Fact_Sales').fetchone()[0]:},")
```

### Resumable

Script band ho gayi? Dobara `python upload_fact_sales.py` — already uploaded rows skip karke aage se continue. Kuch delete karne ki zaroorat nahi.

## 9. Issues aur Solutions (Troubleshooting)

| # | Issue                                 | Cause                                  | Solution                                                                                       |
|---|---------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------|
| 1 | Login failed for user 'Sql_server'    | Wrong password / braces in password    | Raw password (no { }), username without @server                                                |
| 2 | Cannot connect / timeout              | Azure firewall blocking IP             | Azure portal → SQL Server → Networking → Add client IP                                         |
| 3 | "Must declare scalar variable @np"    | Alt+X splits on semicolons             | Use Ctrl+A → Ctrl+Enter for T-SQL with variables                                               |
| 4 | .sql file ko table mein import karna  | Misconception                          | SQL file = SQL editor mein RUN. CSV = table mein import.                                       |
| 5 | 70M load bahut slow (22 min → 0 rows) | Azure tier log-rate throttle           | Tier scale up + FK off + TABLOCK + bada batch                                                  |
| 6 | DBeaver import 3.93M pe toot gaya     | 3.3 GB JDBC transfer breaks on network | T-SQL server-side OR pyodbc chunked upload                                                     |
| 7 | Dims count 209/509 (extra 9)          | Script 01 ke purane 9 rows + CSV add   | No action needed — PKs still unique, FK-safe                                                   |
| 8 | FK constraint violation on insert     | Fact row ki key dim mein nahi          | Generator sirf dim CSV keys use karta hai → impossible                                         |
| 9 | TRUNCATE vs DELETE confusion          | —                                      | TRUNCATE = instant, resets identity. DELETE = slow, log-heavy. Prefer TRUNCATE for bulk clear. |

## 10. PK/FK aur Join Verification

### Complete Verification Query

```
-- 1) Row counts
SELECT 'Fact_Sales' tbl, COUNT_BIG(*) n FROM dbo.Fact_Sales
UNION ALL SELECT 'Dim_Date',COUNT_BIG(*) FROM dbo.Dim_Date
UNION ALL SELECT 'Dim_Store',COUNT_BIG(*) FROM dbo.Dim_Store
UNION ALL SELECT 'Dim_Product',COUNT_BIG(*) FROM dbo.Dim_Product;
-- Expected: 70000000 / ~1461 / ~209 / ~509

-- 2) MAIN JOIN TEST: fact_total == joined_rows = JOINS PERFECT
SELECT COUNT_BIG(*) AS fact_total FROM dbo.Fact_Sales;
SELECT COUNT_BIG(*) AS joined_rows
FROM dbo.Fact_Sales f
JOIN dbo.Dim_Date    d ON f.date_key=d.date_key
JOIN dbo.Dim_Store   s ON f.store_id=s.store_id
JOIN dbo.Dim_Product p ON f.product_id=p.product_id;

-- 3) Orphan check: sab 0 hona chahiye
SELECT
    (SELECT COUNT(*) FROM dbo.Fact_Sales f LEFT JOIN dbo.Dim_Date    d ON f.date_key=d.date_key    WHERE d.date_key
IS NULL) orphan_date,
    (SELECT COUNT(*) FROM dbo.Fact_Sales f LEFT JOIN dbo.Dim_Store   s ON f.store_id=s.store_id    WHERE s.store_id
IS NULL) orphan_store,
    (SELECT COUNT(*) FROM dbo.Fact_Sales f LEFT JOIN dbo.Dim_Product p ON f.product_id=p.product_id WHERE p.product_id
IS NULL) orphan_product;

-- 4) FK trusted? (is_not_trusted = 0 chahiye)
SELECT name, is_not_trusted FROM sys.foreign_keys
WHERE parent_object_id=OBJECT_ID('dbo.Fact_Sales');

-- 5) Business join test
SELECT s.region, COUNT_BIG(*) sales, SUM(f.total_amount) revenue
FROM dbo.Fact_Sales f JOIN dbo.Dim_Store s ON f.store_id=s.store_id
GROUP BY s.region ORDER BY revenue DESC;

-- 6) Sample
SELECT TOP 5 f.sale_id, d.full_date, s.city, p.category, f.total_amount
FROM dbo.Fact_Sales f
JOIN dbo.Dim_Date d ON f.date_key=d.date_key
JOIN dbo.Dim_Store s ON f.store_id=s.store_id
JOIN dbo.Dim_Product p ON f.product_id=p.product_id;
```

### Expected Results

| Check                     | Expected      | Matlab               |
|---------------------------|---------------|----------------------|
| fact_rows                 | 70,000,000    | 70M rows load hue    |
| fact_total == joined_rows | Dono same     | Joins 100% proper    |
| orphan_date/store/product | 0 / 0 / 0     | Koi FK mismatch nahi |
| is_not_trusted            | 0 for all FKs | DB ne validate kiya  |
| Dim PKs dups              | 0 / 0 / 0     | PKs unique           |

## 11. Azure Tier Scale Up/Down

Azure SQL ka service tier directly load speed decide karta hai. Chhota tier = log-rate throttle = slow insert. Load ke time scale up karo, baad mein wapas chhota karo.

### Scale Up (load se pehle)

1. [portal.azure.com](https://portal.azure.com) kholo.
2. **SQL databases** → **EmployeeDatabase** click karo.
3. Left menu → **Compute + storage**.
4. DTU model hai to: Standard S3 (100 DTU) ya upar. vCore model hai to: 4-8 vCores.
5. **Apply / Save** → 2-3 min mein apply hoga.

### Scale Down (load ke baad — paisa bachao)

1. Same page: Compute + storage.
2. Wapas original tier pe laao.
3. Apply.

#### Cost Tip

Scale up sirf load ke waqt karo. 70M load 30-60 min mein ho jata hai bade tier pe. Baad mein chhota tier wapas karo — paisa kam lagega.

## 12. Quick Reference Card

| Kaam              | Tool                 | Command / Action                                                                                          |
|-------------------|----------------------|-----------------------------------------------------------------------------------------------------------|
| Tables banana     | DBeaver              | 01_create_tables.sql → Alt+X                                                                              |
| Dims mock data    | Python               | pip install faker → python generate_mock_dimensions.py                                                    |
| CSV import (dims) | DBeaver              | Table → right-click → Import Data → CSV                                                                   |
| 70M load (fast)   | DBeaver T-SQL        | 02_load_70M_FAST.sql → Ctrl+A → Ctrl+Enter                                                                |
| 70M load (CSV)    | Python               | pip install pyodbc → python upload_fact_sales.py                                                          |
| Partial clear     | DBeaver              | TRUNCATE TABLE dbo.Fact_Sales;                                                                            |
| Verify joins      | DBeaver              | 04_join_verify.sql → Ctrl+A → Ctrl+Enter                                                                  |
| Monitor progress  | DBeaver (new editor) | SELECT SUM(p.rows) FROM sys.partitions p WHERE object_id=OBJECT_ID('dbo.Fact_Sales') AND index_id IN(0,1) |
| Tier scale up     | Azure Portal         | DB → Compute+storage → increase → Apply                                                                   |

### Files Summary

| File                        | Type   | Kaam                                     |
|-----------------------------|--------|------------------------------------------|
| 01_create_tables.sql        | SQL    | 4 tables + PK/FK + seed data banao       |
| generate_mock_dimensions.py | Python | Realistic dim CSVs generate karo         |
| dim_store/product/date.csv  | CSV    | Dimension mock data                      |
| generate_fact_csv.py        | Python | 70M FK-safe fact_sales.csv generate karo |
| 02_load_70M_FAST.sql        | T-SQL  | Server-side 70M load (fastest)           |
| upload_fact_sales.py        | Python | CSV se pyodbc chunked upload (resumable) |
| 03_verify.sql               | SQL    | Counts + PK/FK check                     |
| 04_join_verify.sql          | SQL    | Join correctness verify                  |